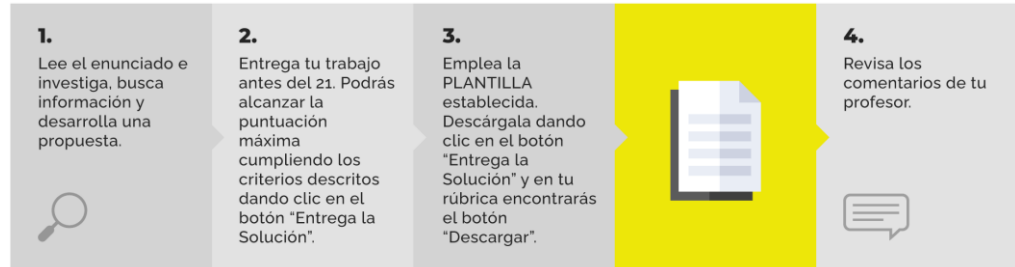


Caso Práctico Unidad 3. Enunciado



Enunciado

Resultado de aprendizaje: Desarrolla e implementa un *backend* funcional y seguro para una **plataforma de e-commerce**, aplicando persistencia de datos en PostgreSQL, control de acceso mediante JWT y estructuras RESTful, con enfoque en **escalabilidad y roles de usuario**.

Contexto.



EcoHome Store consolidó su presencia digital con una **plataforma web en React** conectada a un **backend centralizado en Express.js**. El sistema ya permite **registro, login y consumo del catálogo**, usando **JWT** como autenticación unificada. Además, el módulo de **chat en tiempo real (Socket.IO)** ya está integrado al backend, facilitando comunicación con soporte.

La gerencia detectó dos retos clave en esta nueva etapa:

1. **Expansión al canal móvil:** una proporción importante de clientes usa principalmente el celular. No tener app reduce recurrencia, accesibilidad y fidelización. Por eso se decide crear una aplicación **Flutter** (Android/iOS/web) que **reutilice exactamente el mismo backend**, sin endpoints paralelos.
2. **Trazabilidad interna y responsabilidad sobre los datos:** aunque el CRUD de productos funciona, no queda claro **quién creó cada producto**, lo que limita auditoría y seguimiento. Se necesita asociar cada producto al **usuario autenticado** que lo registra y mostrar métricas simples (ej. "Arturo (14)") que indiquen cuántos productos ha creado cada usuario, visibles en web y móvil y actualizadas al instante.

Resultado esperado del prototipo: demostrar que el ecosistema (React + Flutter) consume el mismo backend con JWT, mantiene sincronización total (productos, chat, usuarios) y agrega trazabilidad y métricas sin romper contratos.

Cuestiones Parte 3

Actividad 1. Integración móvil en Flutter con el backend existente (JWT + catálogo + chat)

Objetivo: demostrar que Flutter consume los mismos servicios que React sin cambios en *endpoints*.

Entregables:

- App Flutter con **login** que consuma el endpoint existente y guarde el **JWT**.
- Pantalla de **catálogo** consumiendo el API de productos (HTTP) con token cuando aplique.
- Integración de **chat Socket.IO** desde Flutter usando el mismo JWT.
- Evidencia:

flujo	completo	en	móvil/emulador
login → listar productos → conectarse al chat → enviar/recibir mensajes.			

Actividad 2. Trazabilidad: asociar productos al usuario creador desde el *backend*

Objetivo: garantizar que cada producto quede ligado al usuario autenticado (auditoría real).

Entregables:

- Cambio en base de datos: relación `products.created_by` (FK a `users.id`) o equivalente.
- Ajuste del endpoint de **crear producto**: tomar el usuario desde el **JWT** y guardar el creador.
- Ajuste de endpoints de consulta: devolver producto + datos mínimos del creador (ej. `username`).
- Evidencia: pruebas (Postman/cURL) mostrando que:
 - al crear un producto con el JWT de “Arturo”, el producto queda asociado a “Arturo”
 - al consultar, aparece el creador.

Actividad 3. UI sincronizada (React + Flutter): mostrar creador y contador dinámico “Usuario (N)”

Objetivo: reflejar en ambos clientes la trazabilidad y una métrica simple de actividad por usuario.

Entregables:

- En React y Flutter: listado de productos mostrando **creador** (`username`).
- Endpoint o estrategia backend para obtener el **contador de productos por usuario** (ej. `/users/me/stats` o incluirlo en respuesta de perfil).
- En React y Flutter: mostrar en la interfaz el usuario autenticado con su contador en formato **Nombre (N)**.
- Actualización dinámica: tras crear un producto, el contador se actualiza inmediatamente en pantalla.
- Evidencia: demo con un usuario creando productos y viendo pasar de “Arturo (14)” a “Arturo (15)” en web y/o móvil.